LanguageName: PLPDI Mini-Project

David Tilman
Student ID: 2763839

October 28, 2026

Repository: `https://github.com/DavidTilman/plpdi-mini-project`
Commit: `TODO`

1 Informal Description

2 Literature Review

3 Formal Syntax and Semantics

3.1 Syntax

$\oplus$	::=	$+ \mid - \mid * \mid / \mid \% \mid < \mid <= \mid == \mid != \mid >= \mid > \mid \mid\mid \mid \&\&$	binary operators
τ	::=	`Nat` $\mid$ `Int` $\mid$ `Real` $\mid$ `Bool`	base types
	$\mid$	$(\tau_1, \ldots, \tau_n) \to \tau$	function type
	$\mid$	R	record type
	$\mid$	$[\tau]$	list
	$\mid$	`EmptyList`	empty list
e	::=	n	literal
	$\mid$	`True` $\mid$ `False`	bool literal
	$\mid$	x	variable
	$\mid$	$-e$	unary negation
	$\mid$	$e_1 \oplus e_2$	binary operation
	$\mid$	$e.f$	field access
	$\mid$	$e(e_1, \ldots, e_n)$	application
	$\mid$	`New` $R(e_1, \ldots, e_n)$	record construction
	$\mid$	`Lambda` $(x_1 : \tau_1, \ldots, x_n : \tau_n) : \tau \Rightarrow e$	lambda
	$\mid$	$e \ ? \ e_1 : e_2$	if expression
	$\mid$	$[e_1, \ldots, e_n]$	list construction
	$\mid$	$e_1 \$ e_2$	list indexing
b	::=	$\{ \, s_1; \ \ldots; \ s_n; \, \}$	block ($n \geq 0$)
s	::=	`Var` $x : \tau = e$	declaration
	$\mid$	`Var` $x : \tau =$ `Read`	declaration from input
	$\mid$	$x := e$	assignment
	$\mid$	`While` (e) b	loop
	$\mid$	`If` (e) b	conditional
	$\mid$	`If` (e) b_1 `Else` b_2	conditional with else
	$\mid$	`Ret` e	return
	$\mid$	`Print` e	print
d	::=	`Record` R $(f_1 : \tau_1, \ldots, f_n : \tau_n)$	record declaration
	$\mid$	`Func` F $(x_1 : \tau_1, \ldots, x_n : \tau_n) : \tau = b$	function declaration
c	::=	$d \mid s$	top-level item
p	::=	$c_1; \ \ldots; \ c_n;$	program

3.2 Type System

3.2.1 Sub-Typing

We have seen that we have 3 numeric types `Nat`, `Int`, `Real`.

However, we see that 3 can be all three at once, -3 can be an `Int` or a `Real`, and 3.5 can only be a `Real`.

This is a problem when we want to make a valid statement such as `Var x : Real = 3;`.

To tackle this we define the notion of sub-typing:

$$\texttt{Nat} \sqsubseteq \texttt{Int} \sqsubset \texttt{Real}$$

This has knock on effects for both the typing rules and implementation of it.

We first define rules so that derivations can determine assign-ability of types:

$$\frac{}{t \sqsubseteq t}\ \text{S-REF} \qquad \frac{t \sqsubseteq t' \qquad t' \sqsubseteq t''}{t \sqsubseteq t''}\ \text{S-TRANS} \qquad \frac{}{\texttt{Nat} \sqsubseteq \texttt{Int}}\ \text{S-NatInt} \qquad \frac{}{\texttt{Int} \sqsubseteq \texttt{Real}}\ \text{S-IntReal}$$

We then define $\mathsf{widest}(\tau_1, \tau_2) \triangleq$ the type with the larger domain: $\texttt{Nat} \leq \texttt{Int} \leq \texttt{Real}$. Widest is only defined for the three number types.

3.2.2 Notation

We define 4 functions:

- $\mathsf{neg}(\tau) \triangleq$ the resulting type when τ is negated:

$$\begin{aligned} \texttt{Nat} &\mapsto \texttt{Int} \\ \texttt{Int} &\mapsto \texttt{Int} \\ \texttt{Real} &\mapsto \texttt{Real} \end{aligned}$$

- $\mathsf{sig}(\oplus, \tau_1, \tau_2) \triangleq$ the resulting type when $\oplus$ is applied to τ_1, τ_2.

  It is a partial function: $\mathsf{sig}(==, \texttt{true}, 3)$ is undefined. Where it is, the operation is not well-formed.

- $\mathsf{join}(\tau_1, \tau_2) \triangleq$ the least type that both τ_1 and τ_2 are assignable to. It collapses the element types of a list literal into a single type (TE-LIST) and types the concatenation of two lists. Like sig it is partial; where it is undefined the list is not well-typed. Taking the first applicable case:

$$\mathsf{join}(\tau_1, \tau_2) = \begin{cases} \texttt{EmptyList} & \tau_1 = \tau_2 = \texttt{EmptyList} \\ [\tau] & \{\tau_1, \tau_2\} = \{\texttt{EmptyList},\ [\tau]\} \\ [\,\mathsf{join}(\tau, \tau')\,] & \tau_1 = [\tau],\ \tau_2 = [\tau'] \\ \tau_1 & \tau_1 = \tau_2 \\ \mathsf{widest}(\tau_1, \tau_2) & \mathsf{widest}(\tau_1, \tau_2) \text{ defined} \\ \text{undefined} & \text{otherwise} \end{cases}$$

- $\mathsf{fld} \triangleq$ the type of a term's field

$$\mathsf{fld}(\tau, f) = \begin{cases} \tau_i & \tau = R,\ \Delta(R) = (\overline{f{:}\tau}),\ f = f_i \\ \texttt{Nat} & \tau = [\tau'],\ f = \texttt{length} \\ \tau' & \tau = [\tau'],\ f = \texttt{head} \\ [\tau'] & \tau = [\tau'],\ f = \texttt{tail} \\ \text{undefined} & \text{otherwise} \end{cases}$$

3.2.3 Expression typing

$$\frac{}{\Gamma \vdash n : \texttt{Nat} \mid \texttt{Real}}\ \text{TE-NUM} \qquad \frac{}{\Gamma \vdash \texttt{True} : \texttt{Bool}}\ \text{TE-TRUE} \qquad \frac{}{\Gamma \vdash \texttt{False} : \texttt{Bool}}\ \text{TE-FALSE}$$

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau}\ \text{TE-VAR} \qquad \frac{\Gamma \vdash e : \tau \qquad \mathsf{neg}(\tau) = \tau'}{\Gamma \vdash -e : \tau'}\ \text{TE-NEGATE}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2 \qquad \mathsf{sig}(\oplus, \tau_1, \tau_2) = \tau}{\Gamma \vdash e_1 \oplus e_2 : \tau}\ \text{TE-OP} \qquad \frac{\Gamma \vdash e : \tau \qquad \mathsf{fld}(\tau, f) = \tau'}{\Gamma \vdash e.f : \tau'}\ \text{TE-FIELD}$$

$$\frac{\Gamma \vdash e : (\tau_1, \ldots, \tau_n) \to \tau \qquad \Gamma \vdash e_i : \tau_i' \qquad \tau_i' \sqsubseteq \tau_i \quad (1 \le i \le n)}{\Gamma \vdash e(e_1, \ldots, e_n) : \tau}\ \text{TE-APP}$$

$$\frac{\Delta(R) = (f_1 : \tau_1, \ldots, f_n : \tau_n) \qquad \Gamma \vdash e_i : \tau_i' \qquad \tau_i' \sqsubseteq \tau_i}{\Gamma \vdash \texttt{New}\ R(e_1, \ldots, e_n) : R}\ \text{TE-RECORD}$$

$$\frac{\Gamma, x_1 : \tau_1, \ldots, x_n : \tau_n \vdash e : \tau' \qquad \tau' \sqsubseteq \tau}{\Gamma \vdash \texttt{Lambda}\,(x_1 : \tau_1, \ldots, x_n : \tau_n) : \tau \Rightarrow e : (\tau_1, \ldots, \tau_n) \to \tau}\ \text{TE-LAMBDA}$$

$$\frac{\Gamma \vdash e : \texttt{Bool} \qquad \Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2 \qquad \tau = \mathsf{join}(\tau_1, \tau_2)}{\Gamma \vdash (e\ \texttt{?}\ e_1\ \ \texttt{:}\ \ e_2) : \tau}\ \text{TE-COND}$$

$$\frac{\Gamma \vdash e_i : \tau_i\ (1 \le i \le n)}{\Gamma \vdash [e_1, \ldots, e_n] : [\mathsf{join}(\tau_1, \ldots, \tau_n)]}\ \text{TE-LIST} \qquad \frac{}{\Gamma \vdash [\,] : \texttt{EmptyList}}\ \text{TE-EMPTY-LIST}$$

$$\frac{\Gamma \vdash e_1 : [\tau] \qquad \Gamma \vdash e_2 : \texttt{Nat}}{\Gamma \vdash e_1 \$ e_2 : \tau}\ \text{TE-LIST-INDEX}$$

TE-NUM, TE-TRUE, TE-FALSE are trivial, except possibly for the omission of an TE-INT rule. This is due to the facts that we cannot know at lex time a whether whole, positive number is an `Int` or `Nat`, and that a negative `Int` literal can be written (and is lexed/parsed as) the unary negation of a `Nat`.

Note that in TE-NUM the type is dependant exclusively on the presence of a decimal point. The widest function can allow a `Nat | Int` to 'become' a `Real`.

TE-VAR, TE-NEGATE and TE-OP are all trivial with the above functions.

TE-FIELD defers to fld, which gives the type of a field on a record (via the record table Δ, defined fully in the operational semantics) or a built-in list field (`length`, `head`, `tail`). fld is undefined on `EmptyList`, so list fields require a known element type.

TE-LAMBDA is the first rule we see extend the typing context Γ. We need to set the type of each parameter in the typing context for the body expression, whose type must be assignable ($\sqsubseteq$) to the lambda's declared return type.

TE-COND uses the join function to allow conditional expressions with differing number types on either side. We use join over widest as the conditional expression is of higher precedence than lists.

TE-List uses sequencial join calls to return the widest type (type can also be a list type) that the list should be.

TE-EMPTY-LIST is trivial.

TE-LIST-INDEX requires a list on the left and a `Nat` index on the right (indices are not widened), and gives the element type.

3.2.4 Statement typing

$$\frac{\Gamma \vdash e : \tau' \qquad \tau' \sqsubseteq \tau}{\Gamma \vdash_{\tau_r} \texttt{Var}\ x : \tau = e \dashv \Gamma, x : \tau}\ \text{TS-DECL} \qquad \frac{\text{Read input is of type } \tau}{\Gamma \vdash_{\tau_r} \texttt{Var}\ x : \tau = \texttt{Read} \dashv \Gamma, x : \tau}\ \text{TS-READ}$$

$$\frac{x : \tau \in \Gamma \qquad \Gamma \vdash e : \tau' \qquad \tau' \sqsubseteq \tau}{\Gamma \vdash_{\tau_r} x := e \dashv \Gamma}\ \text{TS-ASSIGN} \qquad \frac{\Gamma \vdash e : \texttt{Bool} \qquad \Gamma \vdash_{\tau_r} b}{\Gamma \vdash_{\tau_r} \texttt{While}\ (e)\ b \dashv \Gamma}\ \text{TS-WHILE}$$

$$\frac{\Gamma \vdash e : \texttt{Bool} \qquad \Gamma \vdash_{\tau_r} b_1 \qquad \Gamma \vdash_{\tau_r} b_2}{\Gamma \vdash_{\tau_r} \texttt{If}\ (e)\ \texttt{Then}\ b_1\ \texttt{Else}\ b_2 \dashv \Gamma}\ \text{TS-COND} \qquad \frac{\Gamma \vdash e : \tau' \qquad \tau' \sqsubseteq \tau_r}{\Gamma \vdash_{\tau_r} \texttt{Ret}\ e \dashv \Gamma}\ \text{TS-RET}$$

$$\frac{\Gamma \vdash e : \tau}{\Gamma \vdash_{\tau_r} \texttt{Print}\ e \dashv \Gamma}\ \text{TS-PRINT} \qquad \frac{\Gamma_0 \vdash_{\tau_r} s_1 \dashv \Gamma_1 \qquad \Gamma_1 \vdash_{\tau_r} s_2 \dashv \Gamma_2 \qquad \cdots \qquad \Gamma_{n-1} \vdash_{\tau_r} s_n \dashv \Gamma_n}{\Gamma_0 \vdash_{\tau_r} \{\, s_1; \ldots; s_n \,\}}\ \text{TS-BLOCK}$$

Statements do not 'have type', they are 'well-typed'. Thus, their rules do not judge type, they 'return' typing contexts to be used for the next statement.

In these rules, we also pass around a return type τ_r meaning the return type of the block the statement is running in. If the block does not have a return type, as in TP-PROG we say that the return type is $\bullet$, meaning that a type is *not allowed* to return.

In TS-DECL and TS-READ we extend Γ to contain ('know') the type of a variable. We leave TS-READ as an informal rule, as it can only be judged at input time.

TS-ASSIGN asks Γ for the type of x. This is a parse error if not defined. Then we check that we are trying to assign a variable to an expression of an assignable type ($\tau' \sqsubseteq \tau$).

TS-WHILE and TS-COND are trival.

Note that in TS-PRINT, while we can print any type, we must check that the expression e is well-typed.

TS-RET is the only rule to use τ_r. It's premise requires e to be the correct type that is returns.

TS-BLOCK simply evaluates each statement in turn.

3.2.5 Program typing

$$\frac{\Gamma_0 = \overline{f_j : (\tau_{j,1}, \ldots, \tau_{j,k_j}) \to \tau'_j} \qquad \left(\Gamma_0,\ x_{j,1} : \tau_{j,1}, \ldots, x_{j,k_j} : \tau_{j,k_j} \vdash_{\tau'_j} b_j\right)_j \qquad \Gamma_0 \vdash_\bullet \{\, s_1; \ldots; s_n \,\}}{\vdash p \text{ is well-typed}}\ \text{TP-PROG}$$

The P-PROG rule only needs judge if p is well-typed, and does not currently have a Γ.

The rules premises:

1. establish a new typing context Γ_0, where each function f_j has parameter types $(\tau_1, \ldots, \tau_k)$ and returns τ'.
2. check that every function body is well-typed. It says: for each of j functions, start from Γ_0 and extend it with f_j's parameters. Under that context, check that the body b_j returns τ'_j
3. run the top level statements in Γ_0.

3.3 Operational Semantics

3.3.1 Notation

We write:

- $\overline{x_n}$ to mean $x_1, \ldots, x_n$, and $\overline{x : \tau}$ for $x_1 : \tau_1, \ldots, x_n : \tau_n$. $\rho[\overline{x} \mapsto \overline{\ell}]$ abbreviates $\rho[x_1 \mapsto \ell_1, \ldots, x_n \mapsto \ell_n]$, and $\overline{\ell} \notin \mathrm{dom}(\sigma)$ means the ℓ_i are distinct and none is in $\mathrm{dom}(\sigma)$.
- a value $v \ ::= \ n \mid \texttt{True} \mid \texttt{False} \mid R(\overline{v_n}) \mid \mathsf{clo}(\overline{x}, b, \rho) \mid [\,] \mid [\overline{v_n}]$. b in closure is a block body.
- an environment $\rho : \texttt{Var} \to \texttt{Loc}$ to mean a function that maps a variable name `Var` to a location `Loc`.
- that the store $\sigma : \texttt{Loc} \to \texttt{Value}$ maps a location `Loc` to a value `Value`.
- $f[a \mapsto b]$ to mean update f so that it maps a to b.
- $\mathsf{dom}(\sigma)$ to mean the set of locations that σ is defined on.
- the outcome of a *statement* $r \ ::= \ \mathsf{normal} \mid \mathsf{ret}\ v$, where normal is a statement that does not return nor throw an error, and $\mathsf{ret}\ v$ is a statement that returns a value v.
- $\hat{\oplus}$ to mean the result of a binary operation $\oplus$ on two values, v_1, v_2
- Δ for the *record table*, which maps each record name R declared in the program to its fields, so $\Delta(R) = (f_1 : \tau_1, \ldots, f_n : \tau_n)$. Δ is built once by P-PROG and never changes, so we leave it implicit in the judgements.
- $$\mathsf{fld}(v, f) = \begin{cases} v_i & v = R(\overline{v_n}),\ \Delta(R) = (\overline{f{:}\tau}),\ f = f_i \\ n & v = [v_1, \ldots, v_n],\ f = \texttt{length} \\ v_1 & v = [v_1, \ldots, v_n],\ n \geq 1,\ f = \texttt{head} \\ [v_2, \ldots, v_n] & v = [v_1, \ldots, v_n],\ f = \texttt{tail} \\ \text{undefined} & \text{otherwise} \end{cases}$$

3.3.2 Semantic Judgements

We write four kinds of judgement:

- Statements:

  $$\langle s, \rho, \sigma \rangle \Downarrow \langle r, \rho', \sigma' \rangle$$

  A statement s, in environment ρ with store σ evaluates to an outcome r, an updated environment ρ' and store σ'.
- Blocks:

$$\langle b, \rho, \sigma \rangle \Downarrow \langle r, \sigma' \rangle$$

A block b, in environment ρ and store σ evaluates to an outcome r and an updated store σ'.

- Expressions:

$$\langle e, \rho, \sigma \rangle \Downarrow \langle v, \sigma' \rangle$$

An expression e in environment ρ and store σ evaluates to a value and updated store σ'

- Programs:

$$\langle p \rangle \Downarrow \langle \sigma' \rangle$$

A program p runs and ends with an updated store σ'

3.3.3 Expression Semantics

$$\frac{}{\langle n, \rho, \sigma \rangle \Downarrow \langle n, \sigma \rangle}\ \text{E-Num}$$

$$\frac{}{\langle \texttt{True}, \rho, \sigma \rangle \Downarrow \langle \texttt{True}, \sigma \rangle}\ \text{E-True} \qquad \frac{}{\langle \texttt{False}, \rho, \sigma \rangle \Downarrow \langle \texttt{False}, \sigma \rangle}\ \text{E-False}$$

$$\frac{\rho(x) = \ell}{\langle x, \rho, \sigma \rangle \Downarrow \langle \sigma(\ell), \sigma \rangle}\ \text{E-Var} \qquad \frac{\langle e, \rho, \sigma \rangle \Downarrow \langle v, \sigma' \rangle}{\langle -e, \rho, \sigma \rangle \Downarrow \langle \hat{-} v, \sigma' \rangle}\ \text{E-Neg} \qquad \frac{\langle e_1, \rho, \sigma \rangle \Downarrow \langle v_1, \sigma_1 \rangle \qquad \langle e_2, \rho, \sigma_1 \rangle \Downarrow \langle v_2, \sigma_2 \rangle}{\langle e_1 \oplus e_2, \rho, \sigma \rangle \Downarrow \langle v_1 \hat{\oplus} v_2, \sigma_2 \rangle}\ \text{E-Op}$$

$$\frac{\langle e, \rho, \sigma \rangle \Downarrow \langle \texttt{True}, \sigma' \rangle \qquad \langle e_1, \rho, \sigma' \rangle \Downarrow \langle v_1, \sigma'' \rangle}{\langle e\ ?\ e_1 : e_2, \rho, \sigma \rangle \Downarrow \langle v_1, \sigma'' \rangle}\ \text{E-Cond-T} \qquad \frac{\langle e, \rho, \sigma \rangle \Downarrow \langle \texttt{False}, \sigma' \rangle \qquad \langle e_2, \rho, \sigma' \rangle \Downarrow \langle v_2, \sigma'' \rangle}{\langle e\ ?\ e_1 : e_2, \rho, \sigma \rangle \Downarrow \langle v_2, \sigma'' \rangle}\ \text{E-Cond-F}$$

$$\frac{\langle e, \rho, \sigma \rangle \Downarrow \langle v, \sigma' \rangle \qquad \mathsf{fld}(v, f) = v'}{\langle e.f, \rho, \sigma \rangle \Downarrow \langle v', \sigma' \rangle}\ \text{E-FIELD} \qquad \frac{\langle e_i, \rho, \sigma_{i-1} \rangle \Downarrow \langle v_i, \sigma_i \rangle \quad (1 \le i \le n)}{\langle \texttt{New}\ R(\overline{e}), \rho, \sigma_0 \rangle \Downarrow \langle R(\overline{v}), \sigma_n \rangle}\ \text{E-Record}$$

$$\frac{}{\langle \texttt{Lambda}\ (\overline{x : \tau}) : \tau' \Rightarrow e, \rho, \sigma \rangle \Downarrow \langle \mathsf{clo}(\overline{x}, \{\texttt{Ret}\ e;\}, \rho), \sigma \rangle}\ \text{E-Lambda}$$

$$\frac{\langle e, \rho, \sigma \rangle \Downarrow \langle \mathsf{clo}(\overline{x}, b, \rho_\theta), \sigma_0 \rangle \qquad \langle e_i, \rho, \sigma_{i-1} \rangle \Downarrow \langle v_i, \sigma_i \rangle \quad (1 \le i \le n) \qquad \overline{\ell} \notin \mathrm{dom}(\sigma_n) \qquad \langle b, \rho_\theta[\overline{x} \mapsto \overline{\ell}], \sigma_n[\overline{\ell} \mapsto \overline{v}] \rangle \Downarrow \langle \mathsf{ret}\ v, \sigma' \rangle}{\langle e(\overline{e}), \rho, \sigma \rangle \Downarrow \langle v, \sigma' \rangle}\ \text{E-App} \qquad \frac{\langle e_i, \rho, \sigma_{i-1} \rangle \Downarrow \langle v_i, \sigma_i \rangle \quad (1 \le i \le n)}{\langle [\overline{e}], \rho, \sigma \rangle \Downarrow \langle [\overline{v}], \sigma_n \rangle}\ \text{E-LIST}$$

$$\frac{}{\langle [\,], \rho, \sigma \rangle \Downarrow \langle [\,], \sigma \rangle}\ \text{E-EMPTY-LIST} \qquad \frac{\langle e_1, \rho, \sigma \rangle \Downarrow \langle \overline{v}, \sigma_1 \rangle \qquad \langle e_2, \rho, \sigma_1 \rangle \Downarrow \langle i, \sigma_2 \rangle}{\langle e_1 \$ e_2, \rho, \sigma \rangle \Downarrow \langle v_{i+1}, \sigma_2 \rangle}\ \text{E-LIST-INDEX}$$

E-NUM, E-TRUE, E-FALSE are trivial.

In E-VAR we see the first example using ρ and σ. In this rule we retrieve a memory location ℓ from ρ and then see that the return `Value` is the value at $\sigma(\ell)$.

In E-NEG and E-OP we see an example of $\hat{\oplus}$, where after evaluating the expressions on either side, we return the value of the operation applied between the two. Rules for each operation are defined in the appendix (TODO).

The E-COND rules are trivial.

In E-FIELD we require that e evaluates to a record with values $\overline{v}$. This can be achieved with E-VAR. We then require that the given record R has fields $f_1, \ldots, f_n$ and that f is one of those named fields f_i, and return the value of that field v_i.

In E-RECORD we see that, for a record construction $R(\overline{e}) = R(e_1, \ldots, e_n)$, we must evaluate each e_i to a value v_i (mutating and passing a state along the way) and return a new record $R(\overline{v}) = R(v_1, \ldots, v_n)$ with the last state after each expression has been evaluated σ_n.

In E-LAMBDA we must return a closure. The closure $\mathsf{clo}(\overline{x}, \{\texttt{Ret}\ e;\}, \rho)$ represents everything a function needs to run: the parameters $\overline{x}$, the return statement (a lambda is a function that must immediately return an expression on those parameters with its own arguments), and the environment ρ in which it is created (it can see variables defined before it is created).

In E-APP we first evaluate e, which must give a closure $\mathsf{clo}(\overline{x}, b, \rho_\theta)$ (made by E-LAMBDA, or by P-PROG for a `Func`). We then evaluate each argument e_i to a value v_i from left to right, passing the store along as in E-RECORD, ending with σ_n. Each parameter needs somewhere to live, so we pick fresh locations $\overline{\ell}$ that are not already in σ_n; because every call gets new locations, a recursive call cannot overwrite its caller's parameters. Finally we run the body b in the closure's environment ρ_θ extended with $\overline{x} \mapsto \overline{\ell}$, and in the store σ_n extended with $\overline{\ell} \mapsto \overline{v}$. The body runs in ρ_θ, not the caller's ρ, so it sees the variables from where the function was written rather than where it is called (static scoping). The body must finish

with $\mathsf{ret}\ v$: this is the one place an outcome is turned back into a value, and v with the final store σ' is the result of the call. E-APP is the only expression semantic rule that mutates state, as a function has access to global variables.

E-LIST and E-EMPTY-LIST are trivial.

In E-LIST-INDEX we require that e_1 evaluates to a list, and that e_2 evaluates to a number i. Given that $e_1 \Downarrow \overline{v} = v_1, \ldots, v_n$, v_{i+1} is the 0 indexed value.

3.3.4 Statement Semantics

$$\frac{\langle e, \rho, \sigma\rangle \Downarrow \langle v, \sigma'\rangle \qquad \ell \notin \mathrm{dom}(\sigma')}{\langle \texttt{Var}\ x : \tau = e, \rho, \sigma\rangle \Downarrow \langle \mathsf{normal}, \rho[x \mapsto \ell], \sigma'[\ell \mapsto v]\rangle}\ \text{S-VAR}$$

$$\frac{v \text{ of type } \tau \text{ is read from input} \qquad \ell \notin \mathrm{dom}(\sigma)}{\langle \texttt{Var}\ x : \tau = \texttt{Read}, \rho, \sigma\rangle \Downarrow \langle \mathsf{normal}, \rho[x \mapsto \ell], \sigma[\ell \mapsto v]\rangle}\ \text{S-READ}$$

$$\frac{\langle e, \rho, \sigma\rangle \Downarrow \langle v, \sigma'\rangle \qquad \rho(x) = \ell}{\langle x := e, \rho, \sigma\rangle \Downarrow \langle \mathsf{normal}, \rho, \sigma'[\ell \mapsto v]\rangle}\ \text{S-ASSIGN}$$

$$\frac{\langle e, \rho, \sigma\rangle \Downarrow \langle \texttt{False}, \sigma'\rangle}{\langle \texttt{While}\ (e)\ b, \rho, \sigma\rangle \Downarrow \langle \mathsf{normal}, \rho, \sigma'\rangle}\ \text{S-WHILE-F}$$

$$\frac{\langle e, \rho, \sigma\rangle \Downarrow \langle \texttt{True}, \sigma'\rangle \qquad \langle b, \rho, \sigma'\rangle \Downarrow \langle \mathsf{normal}, \sigma''\rangle \qquad \langle \texttt{While}\ (e)\ b, \rho, \sigma''\rangle \Downarrow \langle r, \rho, \sigma'''\rangle}{\langle \texttt{While}\ (e)\ b, \rho, \sigma\rangle \Downarrow \langle r, \rho, \sigma'''\rangle}\ \text{S-WHILE-T}$$

$$\frac{\langle e, \rho, \sigma\rangle \Downarrow \langle \texttt{True}, \sigma'\rangle \qquad \langle b, \rho, \sigma'\rangle \Downarrow \langle \mathsf{ret}\ v, \sigma''\rangle}{\langle \texttt{While}\ (e)\ b, \rho, \sigma\rangle \Downarrow \langle \mathsf{ret}\ v, \rho, \sigma''\rangle}\ \text{S-WHILE-RET}$$

$$\frac{\langle e, \rho, \sigma\rangle \Downarrow \langle \texttt{True}, \sigma'\rangle \qquad \langle b_1, \rho, \sigma'\rangle \Downarrow \langle r_1, \sigma''\rangle}{\langle \texttt{If}\ (e)\ b_1\ \texttt{Else}\ b_2, \rho, \sigma\rangle \Downarrow \langle r_1, \rho, \sigma''\rangle}\ \text{S-IF-T} \qquad \frac{\langle e, \rho, \sigma\rangle \Downarrow \langle \texttt{False}, \sigma'\rangle \qquad \langle b_2, \rho, \sigma'\rangle \Downarrow \langle r_2, \sigma''\rangle}{\langle \texttt{If}\ (e)\ b_1\ \texttt{Else}\ b_2, \rho, \sigma\rangle \Downarrow \langle r_2, \rho, \sigma''\rangle}\ \text{S-IF-F}$$

$$\frac{\langle e, \rho, \sigma\rangle \Downarrow \langle v, \sigma'\rangle}{\langle \texttt{Ret}\ e, \rho, \sigma\rangle \Downarrow \langle \mathsf{ret}\ v, \rho, \sigma'\rangle}\ \text{S-RET}$$

$$\frac{\langle e, \rho, \sigma\rangle \Downarrow \langle v, \sigma'\rangle}{\langle \texttt{Print}\ e, \rho, \sigma\rangle \Downarrow \langle \mathsf{normal}, \rho, \sigma'\rangle}\ \text{S-PRINT}$$

A reminder that a statement can finish normally with normal or return a value with $\mathsf{ret}\ v$.

In S-VAR we evaluate an expression e to a value v, define a fresh location $\ell \notin \mathsf{dom}(\sigma')$ and store the value there. This is notable as the only statement (or rule) that mutates the environment ρ.

We leave S-PRINT as an informal rule. we read v, ensure it is of the correct type τ and store it.

In S-ASSIGN we see a similar structure, only with a mutated store, getting the location to store at through ρ. ρ is unchanged in this rule: the variable must already be declared.

There are three cases for while loops: where the condition is false, or true, or the loop returns early with a $\mathsf{ret}\ v$ statement. S-WHILE-T is recursive, after running the body once, it evaluates to the same statement with a mutated state. S-WHILE-RET returns from the loop early, and passes that value down. Note that the body is a block, its inner environment is thrown away, so that any variable declared in the block is discarded after running.

In the S-IF statements, the condition is evaluated first, then the correct block is selected to be evaluated. The outcome of the block is passed on, so that a `Ret` escapes the `If`.

When a statement returns, we evaluate the expression to a value v and end with a result $\mathsf{ret}\ v$.

We leave S-PRINT as an informal rule. We do need to evaluate $e \to v$ to print it, but the actual printing to console is an external side effect that I do not deem necessary to model.

3.3.5 Block Semantics

$$\frac{}{\langle \{\ \}, \rho, \sigma\rangle \Downarrow \langle \mathsf{normal}, \sigma\rangle}\ \text{B-EMPTY}$$

$$\frac{\langle s_1, \rho, \sigma\rangle \Downarrow \langle \mathsf{normal}, \rho', \sigma'\rangle \qquad \langle \{\ \overline{s}\ \}, \rho', \sigma'\rangle \Downarrow \langle r, \sigma''\rangle}{\langle \{s_1; \overline{s};\}, \rho, \sigma\rangle \Downarrow \langle r, \sigma''\rangle}\ \text{B-SEQ}$$

$$\frac{\langle s_1, \rho, \sigma\rangle \Downarrow \langle \mathsf{ret}\ v, \rho', \sigma'\rangle}{\langle \{s_1; \overline{s};\}, \rho, \sigma\rangle \Downarrow \langle \mathsf{ret}\ v, \sigma'\rangle}\ \text{B-RET}$$

Clearly an empty block simply returns a normal outcome.

B-SEQ takes a list of statements $\overline{s}$ and extracts the first, s_1. If s_1 returns a normal outcome, we take the rest of the block (this can be the empty block), and evaluate that.

B-RET is how a block returns early, we see that the remaining statements $\overline{s}$ are discarded and not run, returning just ret v.

3.3.6 Program Semantics

$$\frac{\begin{array}{c}\text{funcs}(p) = \texttt{Func}\ F_j(\overline{x_j : \tau_j}) : \tau'_j = b_j \quad (1 \le j \le k) \qquad \text{stmts}(p) = \overline{s} \\ \Delta = \{R \mapsto (\overline{f : \tau}) \mid \texttt{Record}\ R\ (\overline{f : \tau}) \in p\} \\ \ell_1, \ldots, \ell_k \text{ distinct} \qquad \rho_g = [F_1 \mapsto \ell_1, \ldots, F_k \mapsto \ell_k] \\ \sigma_0 = [\ell_1 \mapsto \mathsf{clo}(\overline{x_1}, b_1, \rho_g), \ldots, \ell_k \mapsto \mathsf{clo}(\overline{x_k}, b_k, \rho_g)] \\ \langle \{\overline{s}\}, \rho_g, \sigma_0\rangle \Downarrow \langle \mathsf{normal}, \sigma'\rangle\end{array}}{\langle p\rangle \Downarrow \langle \sigma'\rangle}\ \text{P-PROG}$$

The master program rule. Going through all the premises:

1. $\text{funcs}(p) = \texttt{Func}\ F_j(\overline{x_j : \tau_j}) : \tau'_j = b_j \quad (1 \le j \le k)$

   We take all the k functions of the program p and index them. We call the name of the function F_j.

2. $\text{stmts}(p) = \overline{s}$

   We take all the n statements of the program p, in order, as a list $\overline{s} = s_1, \ldots, s_n$.

3. $\Delta = \{R \mapsto (\overline{f : \tau}) \mid \texttt{Record}\ R\ (\overline{f : \tau}) \in p\}$

   We build the record table from the record declarations. Records have no run-time behaviour (they create no locations and no values), so this is the only place they appear. Record values store their fields by position only, so E-FIELD uses Δ to find the position of a field from its name.

4. $\ell_1, \ldots, \ell_k$ distinct

   We choose one new location for each function. The store is empty at this point, so they only need to differ from each other.

5. $\rho_g = [F_1 \mapsto \ell_1, \ldots, F_k \mapsto \ell_k]$

   The global environment ρ_g binds each function name to its location. Functions are values, so a function name is looked up like any other variable with E-VAR.

6. $\sigma_0 = [\ell_1 \mapsto \mathsf{clo}(\overline{x_1}, b_1, \rho_g), \ldots, \ell_k \mapsto \mathsf{clo}(\overline{x_k}, b_k, \rho_g)]$

   The initial store holds a closure for each function. Every closure captures ρ_g itself, so every function can see every other function, including itself. This gives recursion, mutual recursion and calls to functions declared later in the program (*hoisting*). Since ρ_g contains only functions, a function cannot see top-level variables.

7. $\langle \{\overline{s}\}, \rho_g, \sigma_0\rangle \Downarrow \langle \mathsf{normal}, \sigma'\rangle$

   Finally, the top-level statements are run as a single block in ρ_g and σ_0. The block must finish with normal, so a top-level `Ret` has no rule and the program gets stuck. The result of the program is the final store σ'.

4 Implementation

The implementation is written in `C#` .

It has four parts that act in a chain: LEXER → PARSER → TYPE CHECKER → AST INTERPRETER that implement the chain of data TEXT → TOKENS → AST NODES → TYPE CHECKED AST NODES.

The program can then be ran through the interpreter that is the `P_Prog.Execute()` method.

4.1 Architecture

4.1.1 AST Nodes

The implementation of the AST nodes is easy once we have defined grammar and operational semantic rules.

Each grammar rule from the syntax can map exactly to a `C#` record and its own AST node with its operational semantics.

All structural elements of the language can be found in `MiniProject.Language/Structure`.

We begin by defining the base abstract records representing the 8 terminals in our language syntax.

The choice of records over classes is a considered one. In `C#` , records are immutable, that is, once defined it's member variables may not be changed. This allows us to be sure throughout the implementation that external operations cannot change node state or semantics.

A language structure element will has mutability precisely where it is needed.

The abstracts are as follows:

$\oplus$	`abstract record Operation;`	`Operators.cs`
τ	`abstract record Type;`	`Types.cs`
e	`abstract record Expr;`	`Expressions.cs`
b	`abstract record Block;`	`Block.cs`
s	`abstract record Stmt;`	`Statements.cs`
d	`abstract record Decl;`	`Declarations.cs`
c	`abstract record Com;`	`Commands.cs`
p	`abstract record Prog;`	`Program.cs`

For each `abstract` record, we define `sealed` record for each form the terminal can take that stores the data needed to evaluate it.

For example:

```
sealed record OP_Plus : Operation
sealed record T_Record(string Name) : Type
sealed record E_Op(Expr E1, Operation Op, Expr E2) : Expr
sealed record B_Block(IReadOnlyList<Stmt> Statements) : Block
sealed record S_Assign(string Variable, Expr Expr) : Stmt
sealed record D_Record(string Name, IReadOnlyList<Param> Params) : Decl
sealed record C_Stmt(Stmt Stmt) : Com;
sealed record P_Prog(IReadOnlyList<Com> Commands) : Prog
```

Then, when moving to semantics, we must define a model for environment ρ and state (store) σ.

We do this with the following objects:

```
sealed class Ref(Value value) {
    public Value Value { get; set; } = value;
}

sealed record Env(ImmutableDictionary<string, Ref> Vars,
    ImmutableDictionary<string, IReadOnlyList<Param>> Records)
{
    public Ref GetRef(string name) {
        return Vars.TryGetValue(name, out Ref? r)
            ? r
            : throw new Exception("Undefined␣variable␣access.");
    }

    public IReadOnlyList<Param> GetRecord(string name) {
        return Records.TryGetValue(name, out IReadOnlyList<Param>? r)
            ? r
            : throw new Exception("Undefined␣record␣access.");
    }

    public Env Bind(string name, Value val)
        => this with { Vars = Vars.SetItem(name, new Ref(val)) };
}
```

Here, `Ref` acts as our $\sigma : \texttt{Loc} \to \texttt{Value}$, and `Env` acts as our $\rho : \texttt{Var} \to \texttt{Loc}$. We carry σ in ρ for convenience.

Note the difference in mutability here. `Ref` contain a value that can be changed, thus must be implemented as a mutable class. `Env` is strictly immutable, with each binding returning a new `Env`. This is so we can separate environments in our rules.

We then map each of the semantic judgements to a function implemented on every terminal. For this language that is e, s, b, p.

We extend each record:

```
    abstract record Expr {
        public abstract Value Evaluate(Env Env);
    }

    abstract record Stmt {
        public abstract (Outcome, Env) Execute(Env Env);
    }

    abstract record Block {
        public abstract Outcome Execute(Env Env);
    }

    abstract record Prog {
        public abstract void Execute();
    }
```

These abstract functions are where our terms will implement their own semantic judgements.

Statements are the only things that can mutate state, so need to return a new environment along with their outcome.

Each inheritor now must have these rules defined on them. Incidently, it makes it easy to extend the language: one can add a new term, rule, and sealed record implementing the rule.

4.2 Correspondence with the Formal Semantics

Now that the abstracts have defined rules on them, we can inspect the terms, for example:

```
sealed record E_Var(string Name) : Expr {
    public override Value Evaluate(Env Env) {
        return Env.GetRef(Name).Value;
    }
}
```

We see that this closely follows the E-VAR rule: An $e := x$ term takes the variable name, retrieves its location from the environment, and returns the value in the store.

Another example:

```
sealed record S_Cond(Expr Cond, B_Block OnTrue, B_Block OnFalse) : Stmt {
    public override (Outcome, Env) Execute(Env Env) {
        Value cond = Cond.Evaluate(Env);
        if (cond is not V_Bool cond_result)
            throw new Exception("Attempted␣condition␣on␣non␣bool.");
        if(cond_result.Value) {
            return (OnTrue.Execute(Env), Env);
        }
        else {
            return (OnFalse.Execute(Env), Env);
        }
    }
}
```

Here we see an example of a term implementing 2 rules at once. We require that each term implements every judgement on which it is found.

Note that we could have defined a method per semantic rule, but that not only almost doubles the code we need to write, but also adds extra logic for where a term does not actually know which rule it is currently taking the form of (as in the S-WHILE rules).

With all terms, forms, rules, and judgements implemented, we now have a working AST Interpreter: One can define an program using these structural elements and execute it with P-PROG (`P_PROG.Execute()`).

4.3 Type Checker

5 Distinctive-Feature Example Program

6 Reflection on AI and Collaboration

A Remaining Operational Semantics Rules